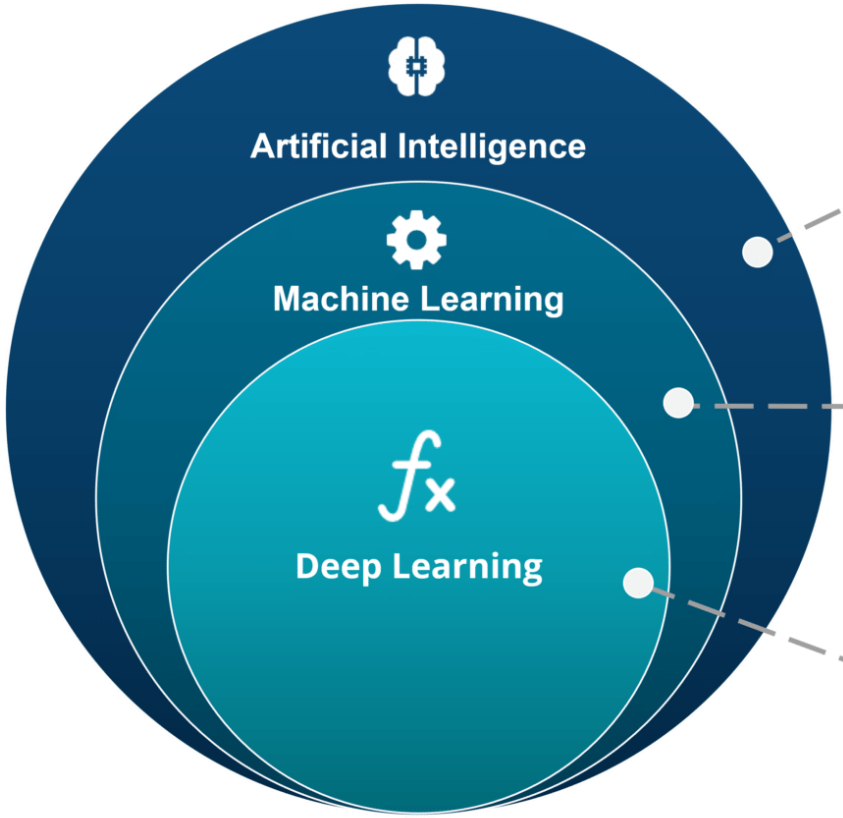
FACULDADE DE
CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE FÍSICA

# Introduction to Machine Leaning

## STATE BASED SEARCH AND GENETIC ALGORITHMS

**Pedro Vieira**
pmv@fct.unl.pt

# Artificial Inteligence




**ARTIFICIAL INTELLIGENCE**
A technique which enables machines to mimic human behaviour

**MACHINE LEARNING**
Subset of AI technique which use statistical methods to enable machines to improve with experience

**DEEP LEARNING**
Subset of ML which make the computation of multi-layer neural network feasible

# Major Events on Artificial Inteligence

| | |
|---|---|
| 1950 | Arthur Samuel learning checkers game |
| 1956 | Dartmouth Workshop coined the term artificial intelligence |
| 1960 | Rosenblatt Perceptron |
| 1970 | First AI Winter |
| 1980 | Expert Systems + Back propagation |
| 1990 | Second AI Winter |
| 1996 | Deep Blue beats the Chess World champion Garry Kasparov |
| 2000 | Big Data, Computing Power, Deep Learning |
| 2015 | AlphaGo + AlphaGo Zero |
| 2017 | Google introduces Transformers "Attention is all you need" |
| 2022 | ChatGPT |

# The 6 major “tribes” in ML

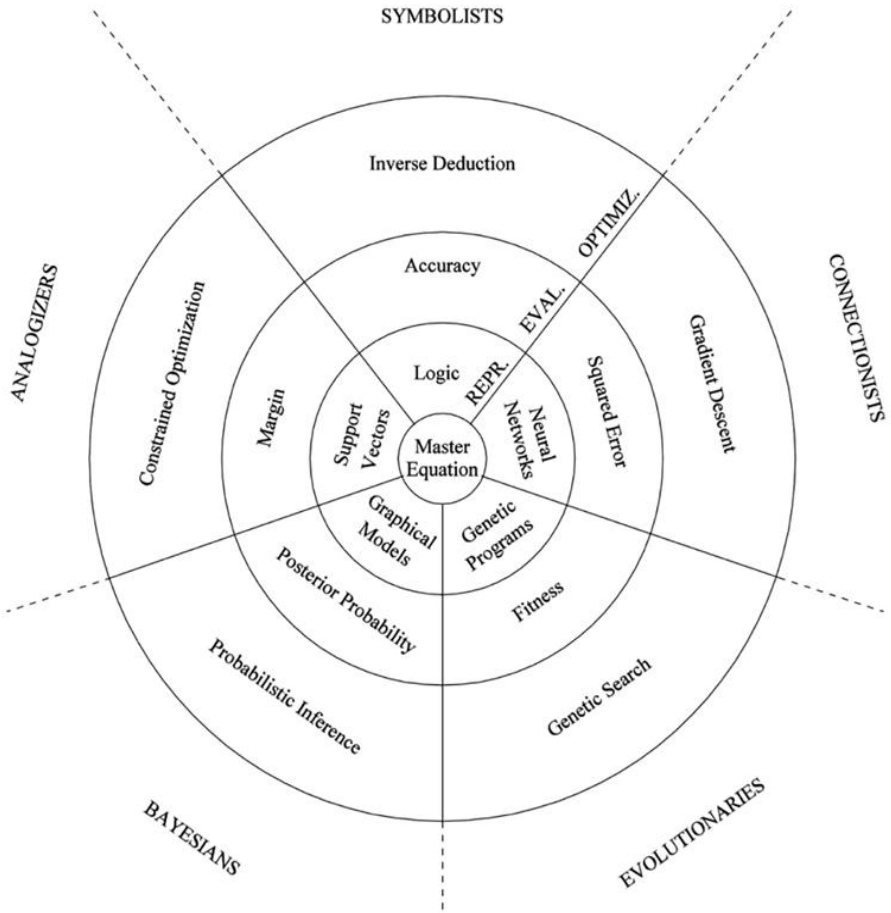
Symbolists
- make machines think like mathematicians

Connectionists
- make machines learn like brains

Analogizers
- make machines learn by analogy, comparing new situations to past ones

Bayesians
- make machines reason like statisticians

Evolutionaries
- make machines evolve like organisms



# Seven Stages of Artificial Intelligence

1. Rule-Based Systems:  AI operates on predefined rules, seen in tasks like business automation and autopilots.
2. Context-Awareness & Retention:  Machines learn from human knowledge in specific domains, updatable for new information, used in applications like chatbots.
3. Domain-Specific Aptitude:  Advanced systems excel in specific fields, potentially surpassing human capabilities.
4. Reasoning Systems: Algorithms possess intellect, intentions, and interact with both machines and humans, approaching commercial viability.
5. Artificial General Intelligence (AGI):  Aspires to human-level intelligence in machines, nearing practical application.
6. Artificial Super Intelligence (ASI):  Algorithms outperform even the smartest humans across all domains, holding potential for addressing complex global challenges.
7. Singularity & Excellency: ASI leads to an extraordinary expansion of human capabilities, enabling seamless mind connectivity and interaction with other life forms.

# AI Principles

Research Issues

1. Research Goal: The goal of AI research should be to create not undirected intelligence, but beneficial intelligence.
2. Research Funding: Investments in AI should be accompanied by funding for research on ensuring its beneficial use, including thorny questions in computer science, economics, law, ethics, and social studies, such as:
   - How can we make future AI systems highly robust, so that they do what we want without malfunctioning or getting hacked?
   - How can we grow our prosperity through automation while maintaining people's resources and purpose?
   - How can we update our legal systems to be more fair and efficient, to keep pace with AI, and to manage the risks associated with AI?
   - What set of values should AI be aligned with, and what legal and ethical status should it have?
3. Science-Policy Link: There should be constructive and healthy exchange between AI researchers and policy-makers.
4. Research Culture: A culture of cooperation, trust, and transparency should be fostered among researchers and developers of AI.
5. Race Avoidance: Teams developing AI systems should actively cooperate to avoid corner-cutting on safety standards.

https://futureoflife.org/ai-principles/

# AI Principles

**Ethics and Values**

1. Safety: AI systems should be safe and secure throughout their operational lifetime, and verifiably so where applicable and feasible.
2. Failure Transparency: If an AI system causes harm, it should be possible to ascertain why.
3. Judicial Transparency: Any involvement by an autonomous system in judicial decision-making should provide a satisfactory explanation auditable by a competent human authority.
4. Responsibility: Designers and builders of advanced AI systems are stakeholders in the moral implications of their use, misuse, and actions, with a responsibility and opportunity to shape those implications.
5. Value Alignment: Highly autonomous AI systems should be designed so that their goals and behaviors can be assured to align with human values throughout their operation.
6. Human Values: AI systems should be designed and operated so as to be compatible with ideals of human dignity, rights, freedoms, and cultural diversity..

https://futureoflife.org/ai-principles/

# AI Principles

**Ethics and Values**

1. Liberty and Privacy: The application of AI to personal data must not unreasonably curtail people's real or perceived liberty.
2. Shared Benefit: AI technologies should benefit and empower as many people as possible.
3. Shared Prosperity: The economic prosperity created by AI should be shared broadly, to benefit all of humanity.
4. Human Control: Humans should choose how and whether to delegate decisions to AI systems, to accomplish human-chosen objectives.
5. Non-subversion: The power conferred by control of highly advanced AI systems should respect and improve, rather than subvert, the social and civic processes on which the health of society depends.
6. AI Arms Race: An arms race in lethal autonomous weapons should be avoided.

# AI Principles

**Ethics and Values**

1. Capability Caution: There being no consensus, we should avoid strong assumptions regarding upper limits on future AI capabilities.
2. Importance: Advanced AI could represent a profound change in the history of life on Earth, and should be planned for and managed with commensurate care and resources.
3. Risks: Risks posed by AI systems, especially catastrophic or existential risks, must be subject to planning and mitigation efforts commensurate with their expected impact.
4. Recursive Self-Improvement: AI systems designed to recursively self-improve or self-replicate in a manner that could lead to rapidly increasing quality or quantity must be subject to strict safety and control measures.
5. Common Good: Superintelligence should only be developed in the service of widely shared ethical ideals, and for the benefit of all humanity rather than one state or organization.

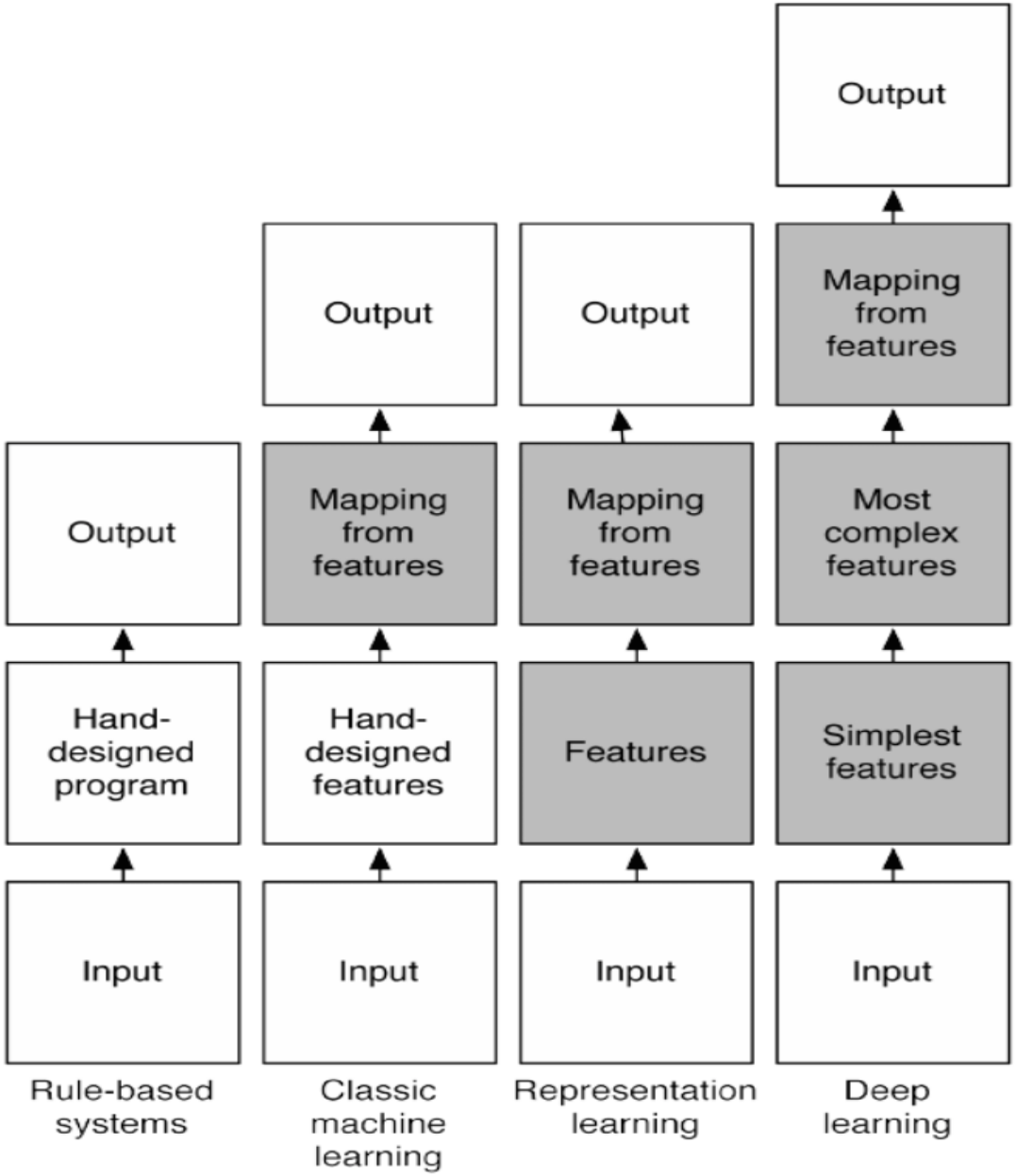
https://futureoflife.org/ai-principles/

# Overview of ML Approaches

A **feature** is an individual, measurable characteristic of the data used as input to a machine learning model.

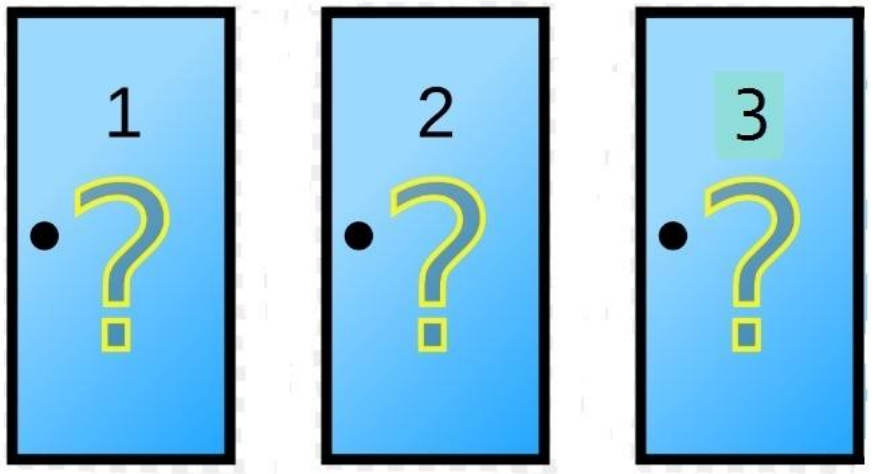
Examples:

- In a housing price model:
  - size, number of rooms, location, age of the house
- In image classification:
  - pixel values, edges, colors, shapes
- Predicting the final velocity of a projectile
  - Initial velocity, Launch angle, Mass of the projectile, Gravitational acceleration, etc …

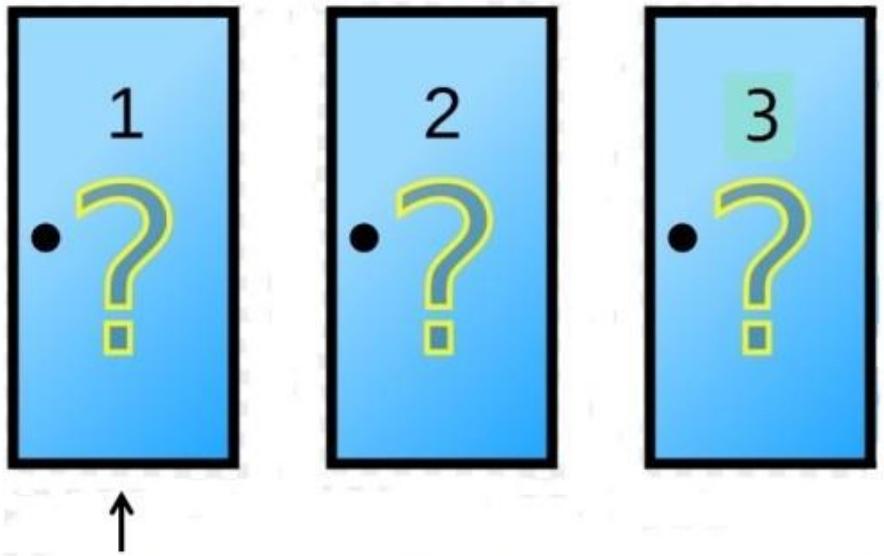


# State-Space Search

# The Human Bias in Probabilistic Thinking
## The Monty Hall Problem



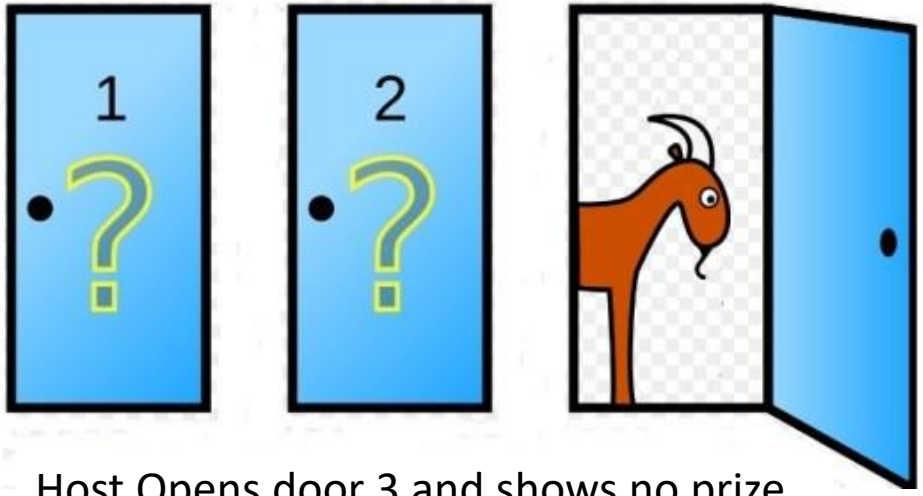

Suppose you are on a game show.
You are given the choice of 3 doors
The prize is behind one of the doors
Each door has the same probability of having the prize (1/3)




You pick door 1. Probability of winning 1/3

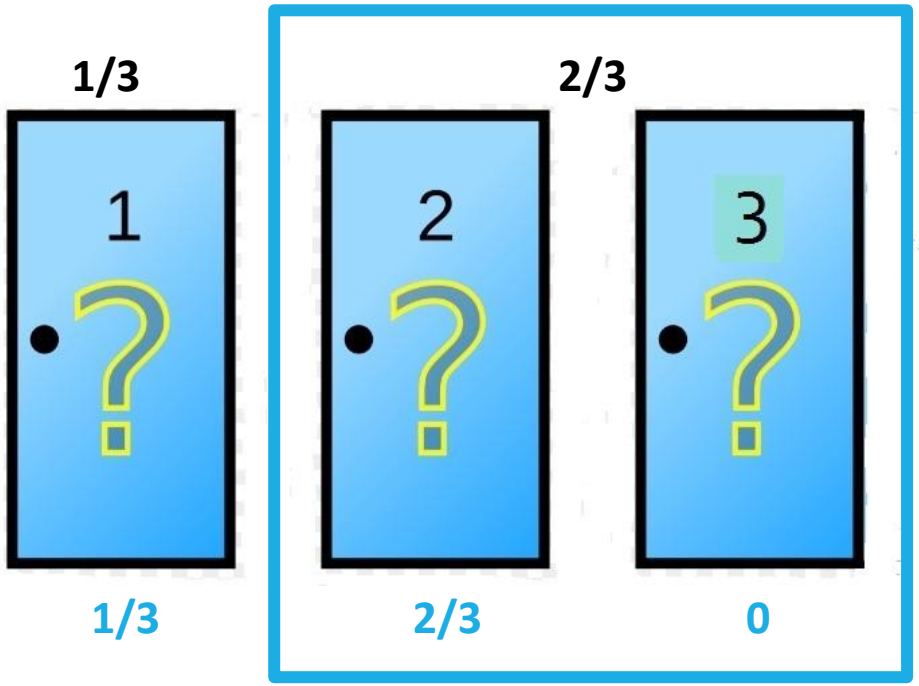
# The Human Bias in Probabilistic Thinking

## The Monty Hall Problem



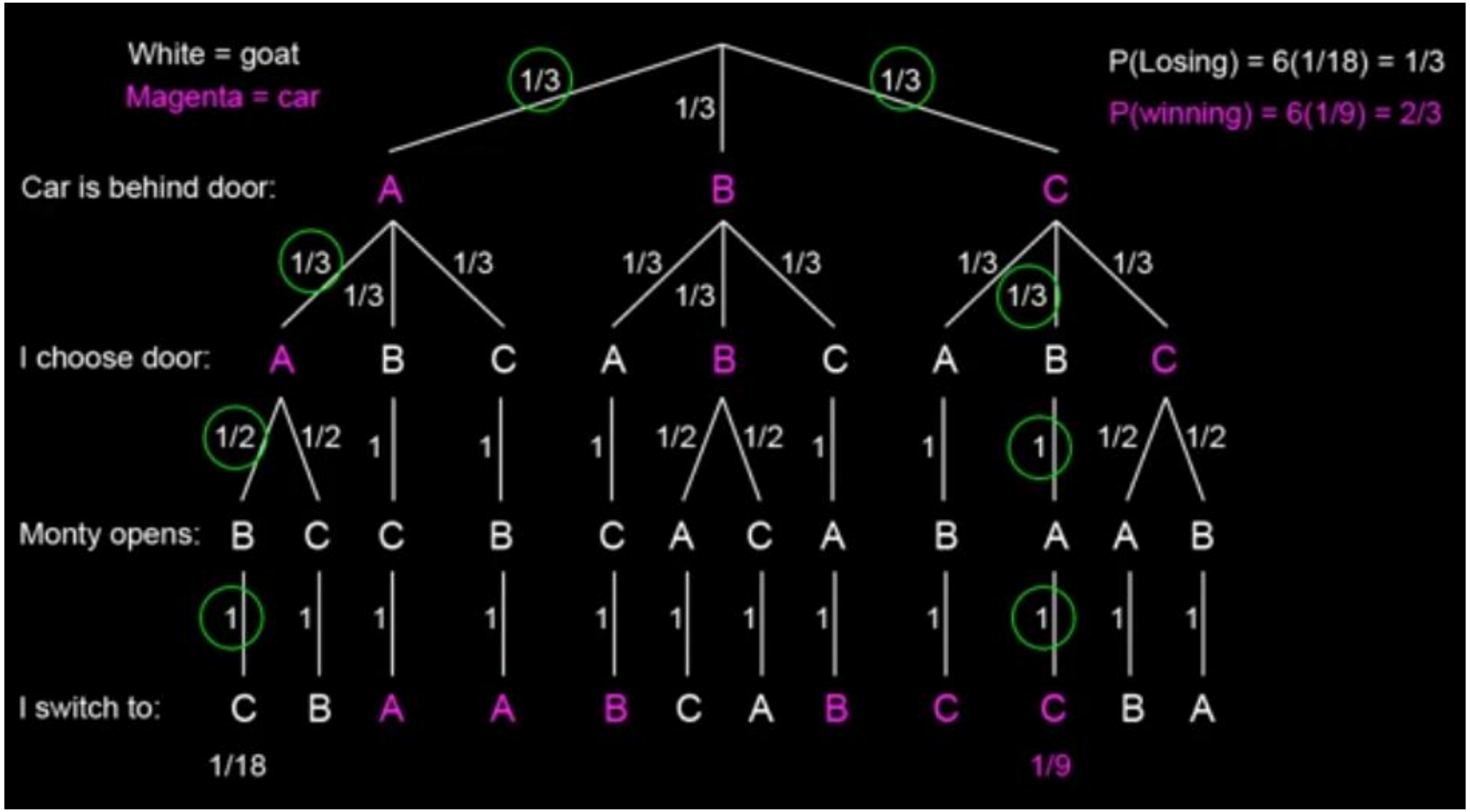

Host Opens door 3 and shows no prize and asks if you want switch door.

Do you switch door?




**This is the base of the Baysian Algorithms**

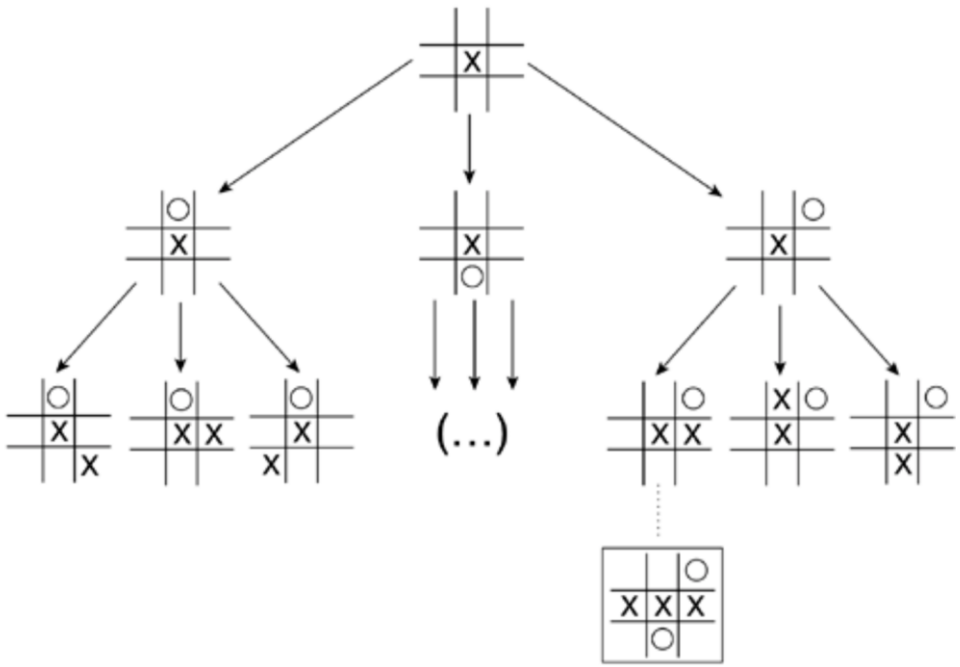
# The Monty Hall Problem
# As a state-space search




See: https://www.youtube.com/watch?v=cphYs1bCeDs

# The State-Based Search

| Component | Meaning |
|---|---|
| **State** | A representation of the system at a moment |
| **Initial state** | Where the search starts |
| **Goal state** | What you want to reach |
| **Actions** | Moves that change the state |
| **Transition model** | Rules for how actions change states |
| **Path cost** | Optional measure of how “expensive” a path is |



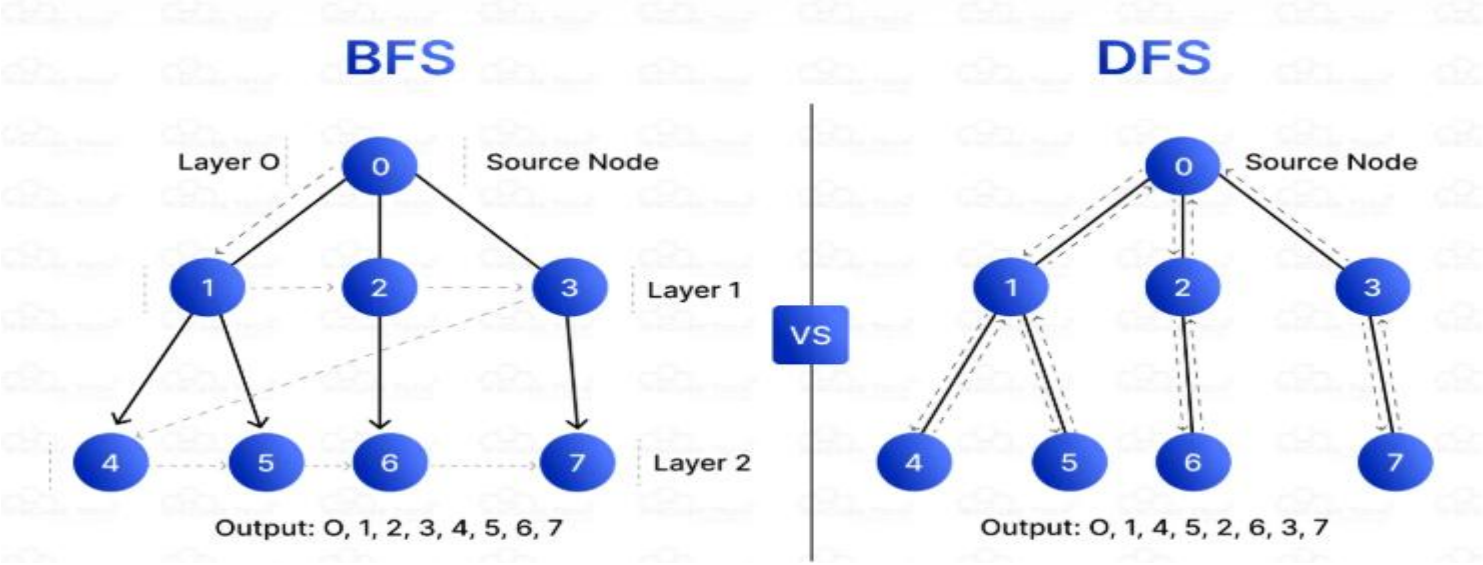
# The State-Based Search
## Main Uninformed Strategies

BFS — Breadth-First Search

- BFS starts at the root and explores all neighbors first, then the neighbors' neighbors, and so on.
- Pros: Guarantees optimal solution
- Cons: High memory requirements

DFS — Depth-First Search

- DFS follows one branch all the way down, then backtracks and tries another.
- Pros: Low memory requirements
- Cons: Slow, may not find a solution for the defined depth



# The State-Based Search
## Informed Strategies – Heuristic Approach

Without anything else BFS o DFS models can explode in complexity

The solution is to give Heuristic rules

- Heuristic rules are not perfect, but good enought to speed up the process
- "A smart guess that guides the search."

Example:

- good heuristic model is an estimation how close node n is to the goal.
- $f(n) = g(n) + h(n)$
  - $f(n)$: efficiency of the node
  - $g(n)$: cost of the path
  - $h(n)$: heuristic model (distance to the solution)

Heuristic search

- **Pros**: Optimizes number of  open nodes
- **Cons**: Needs well informed heuristics

Simple real-world example:

- Imagine you're trying to find a restaurant in a city you don't know.
- BFS: check every street in widening circles
- DFS: pick a street and walk until you hit a dead end
- Heuristic model: "Restaurants are usually near busy areas → walk toward the noise and lights."
- **You're not guaranteed to be right, but you'll probably find food much faster**

# The Knapsack Problem

You are going to travel

You have a number of items that you could take along.

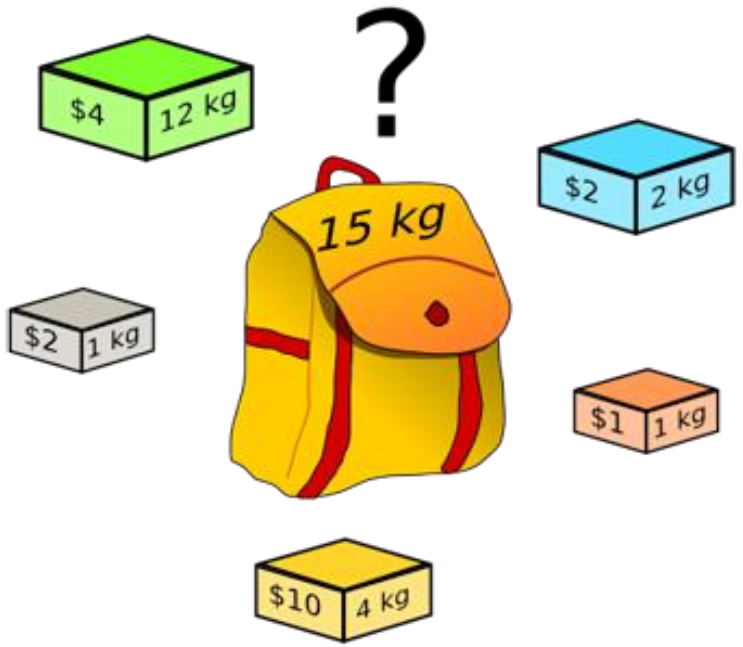
Each item has a weight and a benefit or value.

You can take one of each item at most.

There is a capacity limit on the weight you can carry.

Exemple:

- Item: 1 2 3 4 5 6 7
- Benefit: 5 8 3 2 7 9 4
- Weight: 7 8 4 10 4 6 4
- Knapsack holds a maximum of 22 kilograms
- Fill it to get the maximum benefit




**Classic combinatorial optimization problem**

# The Knapsack Problem
## Bruit Force Solution

The goal is find the combination that:

- Minimise the total weight
- Maximise the value

The number of combinations is:

- $nc = 2^n$
- n the number of itens

Human brain power can find a possible solution for n between 4 and 7.

**The horror of the exponencial behaviour**

**Challenge: write a python script to solve the problem**

| Items (n) | Combinations ($2^n$) | Brute-Force Time |
|---|---|---|
| 5 | 32 | 0.000032 s |
| 10 | 1,024 | 0.0010 s |
| 15 | 32,768 | 0.0328 s |
| 20 | 1,048,576 | 1.05 s |
| 25 | 33,554,432 | 33.6 s |
| 30 | 1,073,741,824 | 17.9 min |
| 35 | 34,359,738,368 | 9.54 h |
| 40 | 1,099,511,627,776 | 12.7 days |
| 45 | 35,184,372,088,832 | 407 days (~1.1 years) |
| 50 | 1,125,899,906,842,624 | 13 days × 100 = **~36 years** |

# The Knapsack Problem
## State-Based Solution

Instead of combinations it uses states.

- For example, is number of itens is 50 and a the weight capacity is 1000, the number of state will be 50.1000 = 50000

| Items (n) | States (n × W) | Approx. Time |
|---|---|---|
| 5 | 5,000 | 0.005 s |
| 10 | 10,000 | 0.010 s |
| 15 | 15,000 | 0.015 s |
| 20 | 20,000 | 0.020 s |
| 25 | 25,000 | 0.025 s |
| 30 | 30,000 | 0.030 s |
| 35 | 35,000 | 0.035 s |
| 40 | 40,000 | 0.040 s |
| 45 | 45,000 | 0.045 s |
| 50 | 50,000 | 0.050 s |

# The Knapsack Problem
## State-Based Solution

Items (weight, value):
- Item 1: weight=2, value=5
- Item 2: weight=3, value=7
- Item 3: weight=4, value=9
- Item 4: weight=1, value=4
- Item 5: weight=5, value=11

Knapsack Capacity: 7

Number of states possible 7*5=35

Each cell of the table represents the value packed

We add the value item to a cell, if:
- The weight capacity of the cell is enough
- More than on item can be present in one cell
- The final value of the cell is higher then before

Capacity

Items

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | | | | | | | | |
| 2 | | | | | | | | |
| 3 | | | | | | | | |
| 4 | | | | | | | | |
| 5 | | | | | | | | |

If no item packed the value will be 0 for all cells

# The Knapsack Problem
## State-Based Solution

Capacity

Items

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 |
| **2** | | | | | | | | |
| **3** | | | | | | | | |
| **4** | | | | | | | | |
| **5** | | | | | | | | |

Adding item 1: weight=2, value=5

Capacity

Items

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 |
| **2** | 0 | 0 | 5 | 7 | 7 | 12 | 12 | 12 |
| **3** | | | | | | | | |
| **4** | | | | | | | | |
| **5** | | | | | | | | |

Adding item 2: weight=3, value=7

# The Knapsack Problem
## State-Based Solution

Capacity

Items

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 |
| **2** | 0 | 0 | 5 | 7 | 7 | 12 | 12 | 12 |
| **3** | 0 | 0 | 5 | 7 | 9 | 12 | 14 | 16 |
| **4** | | | | | | | | |
| **5** | | | | | | | | |

Adding item 3: weight=4, value=9

Capacity

Items

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 |
| **2** | 0 | 0 | 5 | 7 | 7 | 12 | 12 | 12 |
| **3** | 0 | 0 | 5 | 7 | 9 | 12 | 14 | 16 |
| **4** | 0 | 4 | 5 | 9 | 11 | 13 | 16 | 18 |
| **5** | | | | | | | | |

Adding item 4: weight=1, value=4

# The Knapsack Problem
## State-Based Solution

Capacity

Items

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 |
| **2** | 0 | 0 | 5 | 7 | 7 | 12 | 12 | 12 |
| **3** | 0 | 0 | 5 | 7 | 9 | 12 | 14 | 16 |
| **4** | 0 | 4 | 5 | 9 | 11 | 13 | 16 | 18 |
| **5** | 0 | 4 | 5 | 9 | 11 | 13 | 16 | 18 |

Adding item 5: weight=5, value=11

Now we select the best combination:

- Read the column 7 from bottom row
- If previous cell of that column has the same value, this item is discarded
  - Item 5 is out
- Item 4 is in. Remaing weight available: 6
- Item 3 is in. Remaing weight available: 2
- Item 2 is out.
- Item 1 is in. Remaing weight available: 0

Final combination:

- Item 0: weight=2, value=5
- Item 2: weight=4, value=9
- Item 3: weight=1, value=4

# The Knapsack Problem
# Bruit Force vs State-Based Solution

Studie python scripts available

# Genetic Algorithms

# Genetic Algorithms
## Introduction

Directed search algorithms based on the mechanics of biological evolution

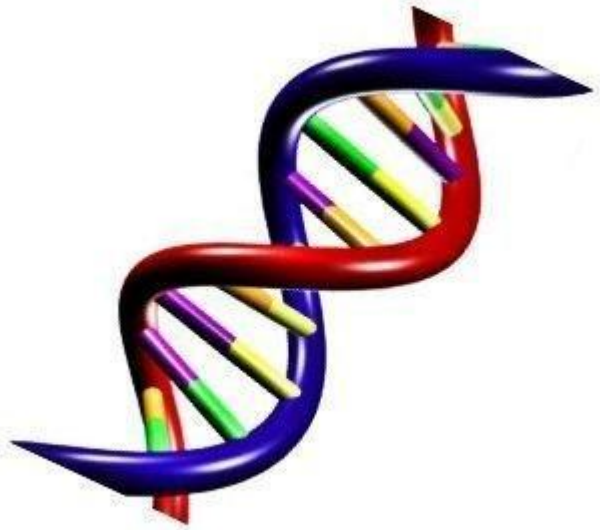
Developed by John Holland, University of Michigan (1970's)

- To understand the adaptive processes of natural systems
- To design artificial systems software that retains the robustness of natural systems

Follows steps inspired by the biological processes of evolution.

Follow the idea of SURVIVAL OF THE FITTEST- Better and better solutions evolve from previous generations until a **near optimal** solution is obtained

This technique are, in many fields, concurrent to the neural networks



# Genetic Algorithms
## How It Works

Each individual of the population, represent a possible solution of the problem and each one is characterized by a chromosome

- Typically, the solutions are represented in binary, but other encodings are also possible

The evolution usually starts from a population of randomly generated individuals and happens in generations.

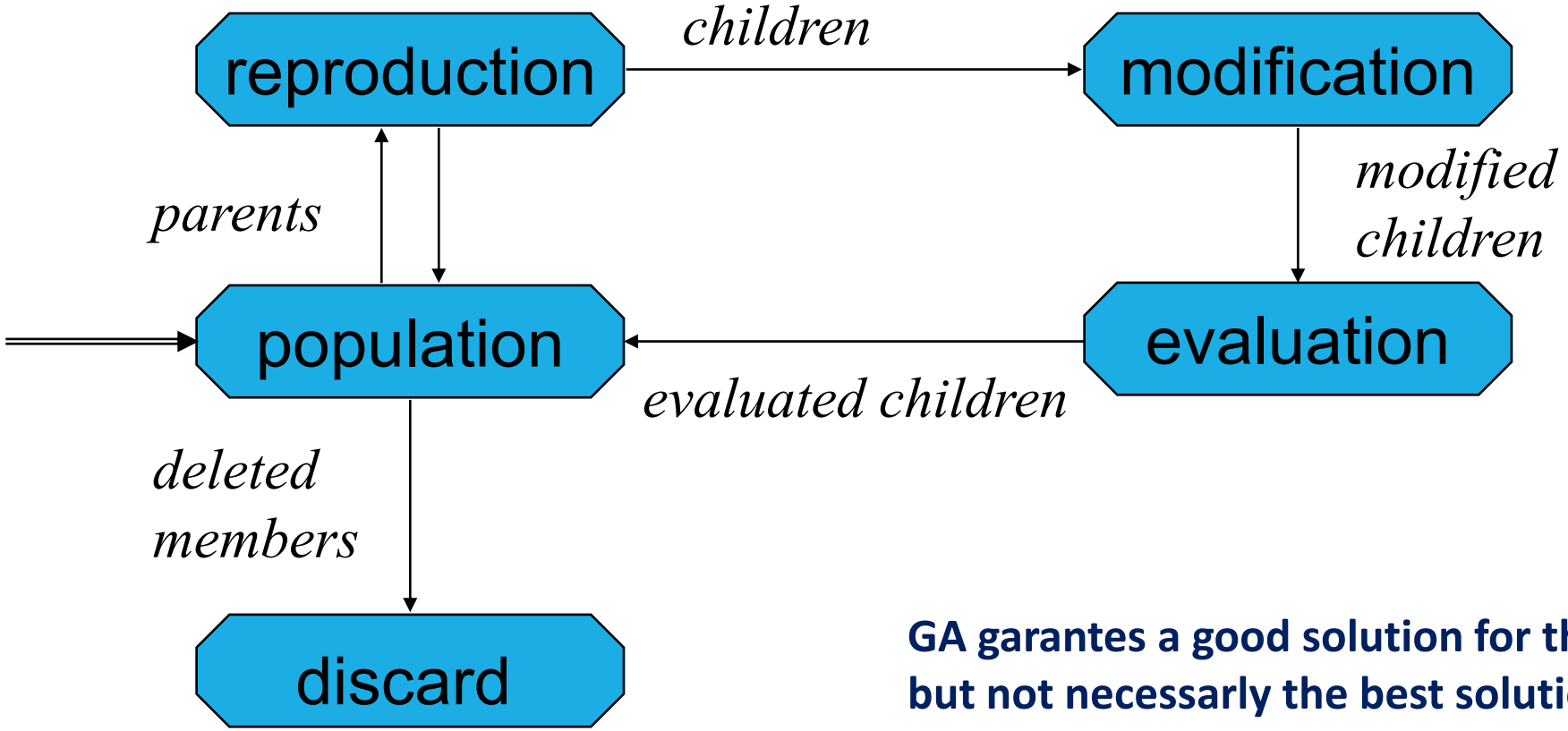
In each generation, the fitness of every individual in the population is evaluated, multiple individuals are selected from the current population (based on their fitness) and modified (recombined and possibly mutated) to form a new population.

The new population is then used in the next iteration of the algorithm.

Commonly, the algorithm terminates when either a maximum number of generations has been produced, or a satisfactory fitness level has been reached for the population.

If the algorithm has terminated due to a maximum number of generations, a satisfactory solution may or may not have been reached.

# Genetic Algorithms
## How It Works




**GA garantes a good solution for the problem, but not necessarly the best solution**

# Genetic Algorithms
## Key Terms

**Individual** - Any possible solution

**Population** - Group of all individuals

**Search Space** - All possible solutions to the problem

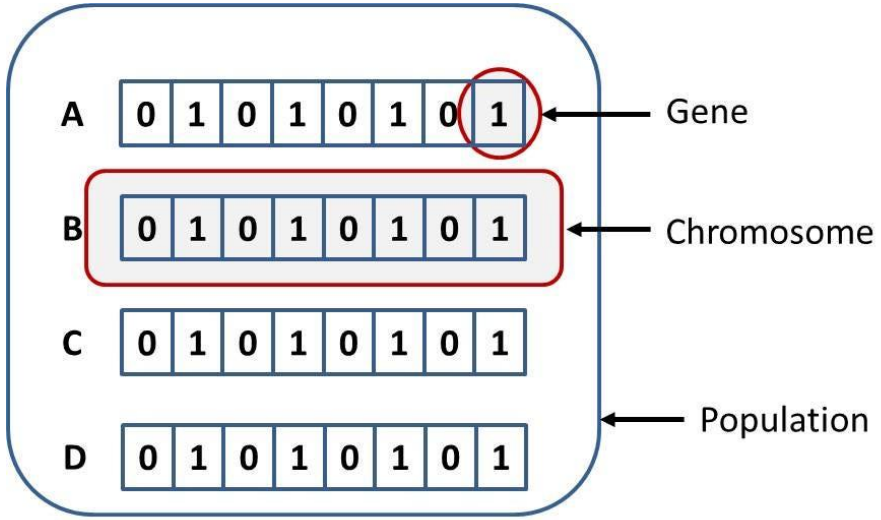
**Chromosome** - Blueprint for an individual

**Trait** - Possible aspect (features) of an individual

**Allele** - Possible settings of trait (black, blond, etc.)

**Locus** - The position of a gene on the chromosome

**Genome** - Collection of all chromosomes for an individual




Gene, Chromosomes and Population

# Genetic Algorithms
## General Operations

**Initialize Population**: generate P hypotheses at random. Typically, 5 to 10 times the number of variables.

**Evaluate**: for each p, compute fitness(p)

**While Max Fitness(h) < Threshold do**

- **Select**: probabilistically select a fraction of the best p's in P. Call this new generation PNew
- **Crossover**: probabilistically form pairs of the selected p's and produce two offsprings by applying the crossover operator. Add all offsprings to Pnew.
- **Mutate**: Choose m% of PNew with uniform probability. For each, invert one randomly selected bit in its representation.
- **Update**: P <- Pnew
- **Evaluate**: for each p in P, compute fitness(p)

**End**: Return the hypothesis from P that has the highest fitness.

# Genetic Algorithms
## Example 1

**Objective**: maximize the number of 1s in a string of length 5, composed only of 1s and 0s, on an entire population

- population size = 4
- chromosome length = 5
- fitness function = no. of genes that are 1

Initial Population (randomly generated)

| | Individue 1 | Individue 2 | Individue 3 | Individue 4 |
|---|---|---|---|---|
| **Chromosome** | 11001 | 00001 | 11111 | 01110 |
| **Fitness** | 3 | 1 | 5 | 3 |

# Genetic Algorithms
## Example 1

**Step 1**

- Select best mates and discard the worst
  - Individual 3 used twice and individual 2 discarded
- Create pairs randomly

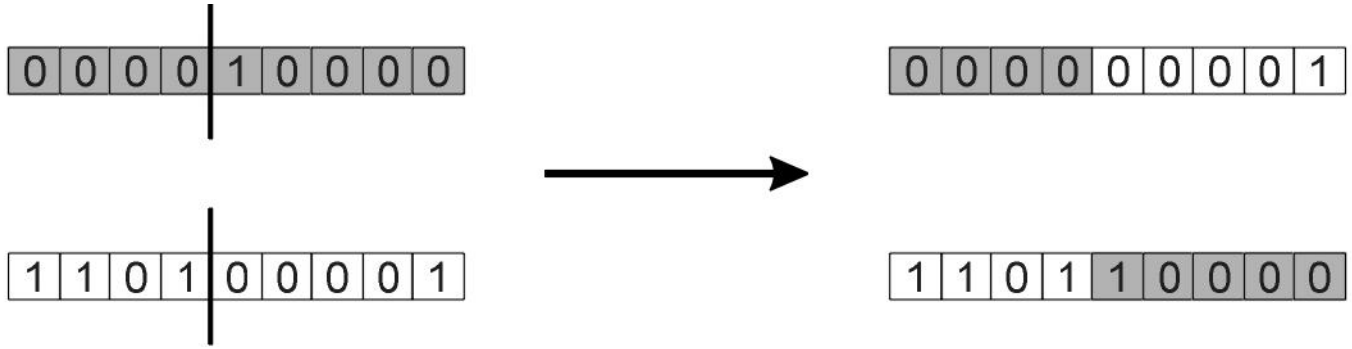
| Pair 1 | Pair 2 |
|---|---|
| 11001 | 11111 |
| 11111 | 01110 |

# Genetic Algorithms
## Example 1 – Cross-over

**Step2**

Using one point cross-over

- Cut point random




| | Pair 1 | Fitness | Pair 2 | Fitness |
|---|---|---|---|---|
| Parent 1 | 110 01 | 3 | 1 1111 | 5 |
| Parent 2 | 111 11 | 5 | 0 1110 | 3 |
| Offsprings 1 | 11011 | 4 | 11110 | 4 |
| Offsprings 2 | 11101 | 4 | 01111 | 4 |

# Genetic Algorithms
## Example 1 – Mutation

**STEP 3**

- Bitwise mutation
- Change 1 bit randomy in some offspings

| | | Fitness | Pair 2 | Fitness |
|---|---|---|---|---|
| Offsprings 1 | 11011 | 4 | 11110 | 4 |
| Offsprings 2 | 11101 | 4 | 01111 | 4 |
| **Offsprings 1** | **01011** | **3** | **11100** | **3** |
| **Offsprings 2** | **11101** | **4** | **01111** | **4** |

**Return to step 1**

- Offspings are the new population

# Genetic Algorithms
## Example 1 – Stop Criterium

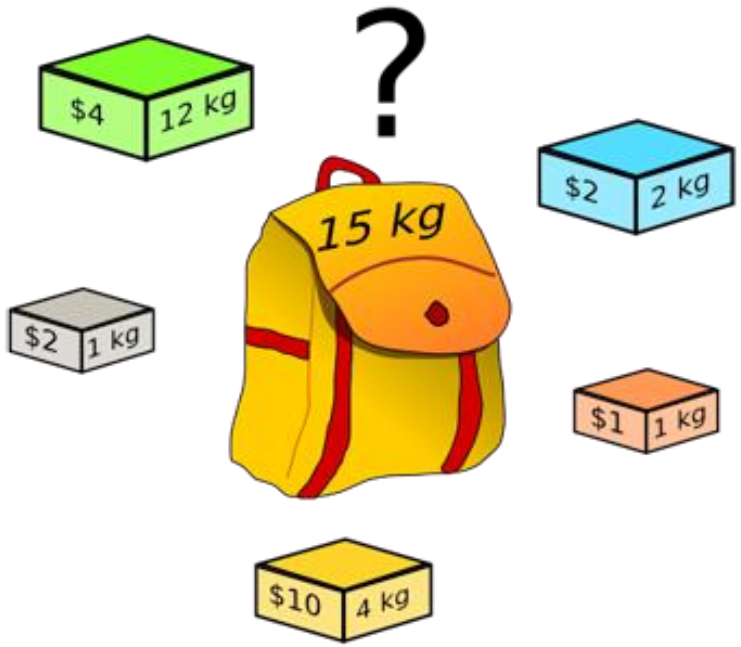
Main loop repeated until stopping criteria met

- for a predetermined no. of generations (ex. 250)
- until a goal is reached (ex. 95%)
- until population converges (low fitness increment between populations)

# Genetic Algorithms
# Example 2 – Knapsack problem

## The GA approach

- Item: 1 2 3 4 5 6 7
- Benefit: 5 8 3 2 7 9 4
- Weight: 7 8 4 10 4 6 4
- Knapsack holds a maximum of 22 kilograms
- Fill it to get the maximum benefit




## Representation

- 7 items -> individual with chromosome of length 7

## Exampe:

- 1100100 -> items 1,2 and 5 included in bag
  - Benefict: 20
  - Weight: 19

# Genetic Algorithms
# Example 2 –fitness criterium

Fitness criterium is the solution with maximum benefit

But if solution overweighs?

how to handle overweight subsets?

- Delete solution?
- Penalize solution?
  - If yes, by how much?
- make correction?
  - Ex: by forcing mutation

So, our criterium will by:

- maximum benefit
- Do not overweight
  - Delete solution that overweight

# Genetic Algorithms
# Example 2 – step by step process

**Step 1**

- Generate a population of n individuals
- Select only the indivuduals that not overweigth.
- Example:
  - **0101010**: Benefit= 19, Weight= 24 **No**
  - **1100100**: Benefit= 20, Weight= 19 **Yes**
  - **0100011**: Benefit= 21, Weight= 18 **Yes**

**Step 2**

- **Select 2 individuals as parents**
  - **Best fitted with some randomless**
  - Generate offsrings by cross-over
  - Generate Mutations

- Example:

| | Chromosome | Weight | Benefict |
|---|---|---|---|
| Parent 1 | 110 0100 | 19 | 20 |
| Parant 2 | 010 0011 | 18 | 21 |
| Offspring 1 | 1100011 | 25 | 26 |
| Offspring 2 | 0100101 | 16 | 21 |

Step 3

- Add valid new offsping to population
- If end condition not satisfied, return to step 2
  - Until stop criterium reach